

Five Attacks on x402 Agentic Payment Protocol

Zelin Li¹, Qin Wang², Zhipeng Wang³, 

¹Ohio State University, US | ²CSIRO, Australia | ³The University of Manchester, UK

Abstract

The *x402* protocol revives the HTTP *402 Payment Required* status code to enable web-native micropayments across APIs, content, and agents. It combines synchronous HTTP authorization with asynchronous blockchain settlement and introduces a cross-layer attack surface absent from conventional web and on-chain payments.

In this paper, we formally analyze x402 and empirically show that it is vulnerable in both design and implementation. We present five concrete attacks that reveal weaknesses in authorization, binding, replay protection, and web-layer handling, showing that x402 is vulnerable across multiple stages of the payment workflow.

We validate these attacks through a reproducible testbed on local chains, Base Sepolia, and live endpoints and further audit three open-source SDKs and endpoints. Our results show that all five attacks are practical and can cause either *unpaid* service or *paid-but-denied* outcomes. We also propose practical mitigations.

Keywords

Blockchain, Agents, Payment, x402, HTTP status code

1 Introduction

The x402 protocol [1] revives the long-reserved HTTP status code 402 Payment Required as a web-native micro-payment handshake. Consider an autonomous AI agent that queries a weather API monetized through *x402* (Fig.1). The interaction begins like an ordinary web request: the agent sends an HTTP GET, and the server replies with 402 Payment Required, quoting a price such as 0.01 USDC on Base. The agent signs a *Payment Payload*. ① It then resends the request with this payload in the X-PAYMENT header. ② The server asks an off-chain facilitator to verify and settle the payment. ③ The facilitator submits the transaction on chain. ④ The server finally grants the requested resource. The result is a seconds-long paid API exchange, without API keys, account setup, or human intervention.

Since its public release by Coinbase in May 2025, the x402 ecosystem has grown rapidly: open-source SDKs exist in TypeScript, Python, Go, and .NET; a discovery layer (*Bazaar*) catalogs over 13,000 registered resource servers; and the protocol is cited as the payment backbone for Agent-to-Agent (A2A) commerce [2, 3].

Yet the apparent simplicity of x402 masks a *cross-layer trust* gap. The HTTP layer operates synchronously: a request either succeeds or fails within milliseconds. The blockchain layer, by contrast, offers only *probabilistic finality*: a transaction included in block *b* may be reverted by a reorganization of depth *k*. x402 bridges these two layers through an off-chain facilitator whose correctness is neither enforced by the protocol nor verifiable by the client.

This mismatch spans the phases and parties in the payment workflow, creating a large attack surface. Even before payment begins, the discovery layer may steer agents toward adversarial resource servers. Once the agent enters the payment flow, the X-PAYMENT

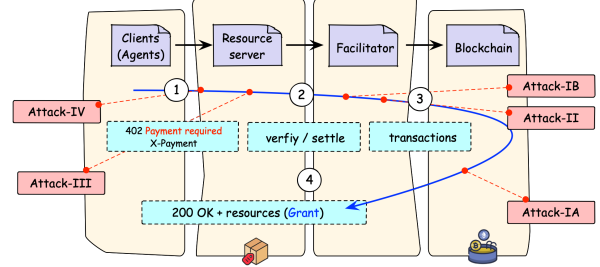


Figure 1: Simplified x402 workflow and our attack points. ① The client resends the request with X-PAYMENT. ② The server asks the facilitator to verify and settle. ③ The facilitator submits the transaction on chain. ④ The server grants the resource after settlement. Our five attack points are in red.

header becomes a bearer-style payment capability that can be replayed or cached by intermediaries (①). The server then relies on an off-chain facilitator for verification and settlement decisions, so a non-final or incorrect verification signal may still trigger access (②). The settlement path may fail to bind the on-chain caller, facilitator, resource, and service decision into one atomic object (③). Finally, after the server releases the resource, later settlement failure or cache reuse cannot be naturally rolled back (④).

x402 should be treated as a cross-layer security protocol, not merely as an HTTP payment wrapper. However, to our knowledge, x402 has received rare prior formal security analysis. Industry documentation focuses mainly on integration guides and happy-path examples, while the protocol specification omits several security-critical details. Existing academic work on payment channels [4, 5], probabilistic micropayments [6], service execution [7], and API security [8] does not fully address x402’s boundaries, where synchronous web authorization is tightly coupled with asynchronous blockchain settlement.

In this paper, we investigate the following question:

What are the concrete security risks in x402’s current design and implementations, and how can they be mitigated without sacrificing its web-native usability?

We answer this question through four attack classes, a targeted implementation audit, and live testnet validation (Table 1):

- **Attack I: settlement-path inconsistencies.** We study failures between HTTP access control and on-chain settlement. Attack I-A shows that optimistic execution may grant resources before payment is final. Attack I-B shows that caller-unbound settlement lets an observer consume the payment authorization before the legitimate facilitator.
- **Attack II: replay and idempotency.** We analyze how a reusable X-PAYMENT payload can produce multiple HTTP-layer grants when the server does not atomically record a payment identity before releasing the resource.

Table 1: Summary of our attacks on x402 payment protocols. RGP = revert-grant probability; DGR = duplicate-grant rate.

Attack	Mechanism	Impact	Experimental Results
I-A: Revert-grant	Grant before finality	Unpaid service after settlement failure	RGP ₀ 5.18%; Byzantine: 100%
I-B: Settlement preemption	Caller-unbound Permit2 settle()	Payment without service	Base Sepolia: preemption yields HTTP 402
II: Replay / Idempotency	No idempotency/binding	One payment reused for many grants	DGR = n ; live: 248 grants/payment
III: Header / Proxy confusion	Header ambiguity; caching	Verification confusion and paid-content leak	nginx leak: 100%; live CDN leak
IV: Server-selection attacks	Metadata gaming + Sybils	Adversarial capture of agent traffic	One crafted server: 71.8%; five Sybils: 60.2%

- **Attack III: web-layer handling.** We examine how ordinary HTTP infrastructure interacts with payment-gated content. In particular, we validate cache leakage as an end-to-end failure and discuss header ambiguity as a deployment-dependent parser risk.

- **Attack IV: server selection.** We study how agent-side discovery can be manipulated before the payment protocol even begins. We empirically validate metadata manipulation and Sybil flooding in Bazaar-style server selection, showing that adversaries can bias agents toward malicious paid endpoints.

These attacks cover the main stages of the x402 workflow: discovery, payment presentation, facilitator verification, on-chain settlement, and final resource delivery. We evaluate them across controlled, testnet, and implementation-level studies.

Specifically, we summarize our contributions as follows.

- We present a *formalized security abstraction for x402*. We model the main entities, trust boundaries, workflow stages, and settlement assumptions, and abstract four security properties from this model: authorization soundness, payment–service correspondence, replay resistance, and facilitator k -atomicity. These properties provide a structured basis for analyzing where x402 holds, where it fails, and how different execution policies change the resulting risk (Theorems 6–9, §2.2–2.3).
- We characterize *five attacks across four classes* with measurable metrics. As shown in Table 1, the attacks include settlement-path inconsistencies including revert-grant under optimistic execution (Attack I-A) and unauthorized settlement preemption (Attack I-B), replay/idempotency attacks (Attack II), header/proxy confusion (Attack III), and agent server-selection manipulation (Attack IV), each specified with violation events, adversary capability sets, and quantitative indicators (§3).
- We implement a *reproducible testbed* and run over 25,000 payment requests across 48 configurations on Hardhat/Anvil and Base Sepolia, supplemented by validation on four production x402 endpoints. The experiments confirm all attack classes, including RGP₀ up to 5.18% with honest facilitators, DGR = n without idempotency, on-chain settlement preemption, production CDN cache leakage, and agent selection bias up to 71.8% under metadata manipulation and 60.2% under 5-Sybil flooding. We report 95% Wilson score confidence intervals for rate metrics (§4).
- We conduct a *systematic multi-implementation audit* of three open-source x402 SDKs and four live endpoints, identifying 11 vulnerabilities across five classes. These include grant-before-settle behavior in a third-party Python SDK, missing resource-identifier binding, fire-and-forget settlement, and absent Cache-Control headers, with severity ratings (§4.6).

- We propose *actionable mitigations*, such as canonical encoding, two-phase settlement, mandatory idempotency with resource binding, and caching hygiene, and report findings via coordinated vulnerability disclosure (§6).

Responsible disclosure. We privately reported our findings to Coinbase via HackerOne (#3679163, #3679179, #3679220), including issues on x402 permit settlement, Bazaar discovery registration and resource-identifier binding.

2 x402 Models and Protocols

This section presents the protocol flow, assumptions, security properties, and our theoretical results.

2.1 x402 Overview

The x402 protocol reinterprets the HTTP status 402 Payment Required as a web-native payment handshake. A typical exchange proceeds in three phases:

- (1) **Request and quote.** The client sends an ordinary HTTP request, and the server replies with 402 Payment Required and a *Payment Requirements* object that quotes the payment terms.
- (2) **Payment presentation.** The client constructs a signed *Payment Payload* and resends the request via the X-PAYMENT header.
- (3) **Verification, settlement, and grant.** The server asks the facilitator to verify and settle the payment, the facilitator submits the transaction on chain, and the server grants the protected resource once settlement succeeds.

2.2 System Model

We first define the entities, messages, and protocols.

Notation and entities. We model an **x402 system** as a tuple $\mathcal{S} = (C, \mathcal{R}, \mathcal{F}, \mathcal{B}, \mathcal{N}, \mathcal{T}, \lambda)$. Here, $C = \{C_i\}$ denotes the set of *clients*; $\mathcal{R} = \{R_j\}$ the set of *resource servers* that host protected resources and issue HTTP 402 responses; $\mathcal{F} = \{F_t\}$ the set of optional *facilitators* that verify payments and/or coordinate on-chain settlement; $\mathcal{B}$ the *blockchain ledger* used for settlement, modeled as an append-only sequence of blocks with probabilistic finality; $\mathcal{N}$ the *network and communication infrastructure*, including HTTP/TLS channels, proxies, caches, and blockchain mempools; $\mathcal{T}$ the global logical time domain; and λ the security parameter.

We write $(x \xrightarrow{m} y, t)$ to denote that at time t entity x sends message m to entity y over $\mathcal{N}$.

x402 messages. We define the primary structured messages exchanged in an x402 protocol execution:

$PR := \langle \text{resource_id}, \text{amount}, \text{token}, \text{chain_id}, \text{receiver}, \text{expiry}, \text{meta} \rangle$

$PP := \langle \text{payment_id}, \text{payer_addr}, \text{amount}, \text{chain_id}, \text{nonce}, \text{ts}, \sigma \rangle$

Here, PR denotes *PaymentRequirements* and PP denotes *Payment-Payload*. The symbol σ denotes a digital signature produced by the payer over a canonical serialization of the payload fields. We let $\text{pay_id} = PP.\text{payment_id}$ denote a unique identifier derived by the client or facilitator.

Blockchain model. The blockchain $\mathcal{B}$ exposes the following primitives: Broadcast(tx), which submits a transaction to the mempool; Include(tx, b), which records that tx appears in block b at height h ; Confirmations(tx, t), which returns the number of confirmations observed for tx at time t ; Final(tx, k), which checks whether tx has reached the k -confirmation finality threshold; and Reorg(b), which captures a chain reorganization that may remove previously included transactions from the canonical chain. We model inclusion and finality with probabilistic latency: for a broadcast at time t_0 , inclusion time is a random variable T_{inc} and finality occurs at $t_0 + T_{\text{fin}}$. We denote the block interval as T_b .

Definition 1 (Chain Finality Bound). The blockchain $\mathcal{B}$ satisfies the k -deep finality property if $\Pr[\text{Reorg}(\text{depth} \geq k)] \leq \epsilon_{\text{chain}}(k)$, where $\epsilon_{\text{chain}}(k)$ is a monotonically decreasing function of k . For chains with exponential finality decay, $\epsilon_{\text{chain}}(k) = e^{-\alpha k}$ for a chain-specific constant $\alpha > 0$.

x402 protocol. An x402 *execution trace* $\mathcal{E}_{x402}$ represents the ordered interactions among the client (C), resource server (R), facilitator (F), and blockchain ($\mathcal{B}$). Specifically, $\mathcal{E}_{x402}$ is a finite sequence of events: $\mathcal{E} = (e_1, e_2, \dots, e_n)$, $e_i \in \mathcal{M} \cup \mathcal{O}$, where $\mathcal{M}$ are off-chain messages (HTTP requests/responses, facilitator API calls) and $\mathcal{O}$ are on-chain observations (transactions broadcast, included, reverted).

Following the official x402 protocol specification [1], we decompose the canonical trace into three subprotocols:

$$\Pi_{x402} = \Pi_{\text{rq}} \circ \Pi_{\text{pp}} \circ \Pi_{\text{vs}},$$

where Π_{rq} is the request-and-quote subprotocol, Π_{pp} is the payment-presentation subprotocol, and Π_{vs} is the verification-and-settlement subprotocol that culminates in a resource grant.

Π_{rq} : **Request-and-quote subprotocol.** The client issues an ordinary web request and receives the quoted payment terms.

- $C \xrightarrow{\text{GET /api}} R$: the client requests a protected resource.
- $R \xrightarrow{\text{402 Payment Required}} C$: the server returns one or more payment requirements that quote the amount, token, receiver, and expiry.
- C : the client parses PR and selects an admissible payment option.

Π_{pp} : **Payment-presentation.** The client signs a Payment Payload PP and resends the request in X-PAYMENT header.

- C : the client selects a payment method and constructs PP .
- $C \xrightarrow{\text{X-PAYMENT: } PP} R$: the client resends the request with PP .

Π_{vs} : **Verification, settlement, and grant.** The server asks the facilitator to verify and settle, the transaction is submitted on chain, and the resource is granted on success.

- $R \xrightarrow{\text{/verify } \langle PP, PR \rangle} F$: the server asks the facilitator to verify the presented payment, or equivalently verifies locally.
- $F \xrightarrow{\text{Verifi. Response}} R$: the facilitator returns its validation verdict.
- $R \xrightarrow{\text{/settle } \langle PP, PR \rangle} F$: if verification succeeds, the server asks the facilitator to settle.
- $F \xrightarrow{\text{tx}_{pp}} \mathcal{B}$: the facilitator submits the settlement transaction tx_{pp} on-chain.
- $\mathcal{B} \xrightarrow{\text{confirm}(k)} F$: the chain confirms the transaction to depth k .
- $F \xrightarrow{\text{settled}} R$: the facilitator reports settlement status to the server.
- $R \xrightarrow{200 \text{ OK, X-PAYMENT-RESPONSE}} C$: the server grants the requested resource after the settlement and emits the terminal application-layer response.

2.3 Threat Model

We then specify the adversarial capabilities and our assumptions.

We consider a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ that interacts with the x402 system $\mathcal{S} = (\mathcal{C}, \mathcal{R}, \mathcal{F}, \mathcal{B}, \mathcal{N}, \mathcal{T}, \lambda)$ as defined in §2.2. The adversary may possess different classes of capabilities, denoted $\mathcal{A}_X$ for domain $X \in \{\mathcal{N}, \mathcal{C}, \mathcal{F}, \mathcal{B}, \mathcal{R}\}$.

Adversarial capabilities are defined as follows:

- **Network adversary $\mathcal{A}_{\mathcal{N}}$:** Can delay, drop, reorder, or replay HTTP messages, consistent with the network layer of $\mathcal{N}$. TLS channels guarantee message integrity and confidentiality, but $\mathcal{A}_{\mathcal{N}}$ can control timing and scheduling.
- **Client adversary $\mathcal{A}_{\mathcal{C}}$:** May corrupt an arbitrary subset $C_{\mathcal{A}} \subseteq \mathcal{C}$, enabling generation of malformed or replayed Payment Payloads, or deviation from honest protocol behavior.
- **Facilitator adversary $\mathcal{A}_{\mathcal{F}}$:** May corrupt facilitators $\mathcal{F}_{\mathcal{A}} \subseteq \mathcal{F}$, causing Byzantine behavior such as returning false verifications or prematurely asserting settlement.
- **Blockchain adversary $\mathcal{A}_{\mathcal{B}}$:** Can observe and manipulate the public mempool; may inject, reorder, or replace transactions within consensus constraints. Cannot violate the chain finality bound (Definition 1): $\Pr[\text{reorg depth} > k] \leq \epsilon_{\text{chain}}(k)$.
- **Server adversary $\mathcal{A}_{\mathcal{R}}$:** Resource servers are assumed *honest-but-curious*: they follow the protocol but may record observable data. Fully malicious servers are discussed in §6.

Structural assumptions. We make two structural assumptions:

- **Blockchain independence.** Chain reorganizations depend only on the honest/adversarial mining or staking power and block-propagation delays. Conditioned on the broadcast transactions, whether $\text{Reorg}(\text{depth} \geq k)$ occurs is independent of the facilitator's internal state and HTTP-layer events.
- **Facilitator determinism.** An honest facilitator F 's report $F.\text{reportFinal}(tx, t)$ is a deterministic function of its local view of $\mathcal{B}$ at time t . A Byzantine facilitator is modeled as an arbitrary PPT machine.

Cryptographic assumptions. All cryptographic primitives (e.g., digital signatures, hash functions) used in the system are secure. The adversary can only break them with negligible probability in λ .

Composed adversaries. Adversaries may combine capabilities: $\mathcal{A} = \bigcup_{X \in \mathbb{A}} \mathcal{A}_X$ where $\mathbb{A} \subseteq \{\mathcal{N}, \mathcal{C}, \mathcal{F}, \mathcal{B}, \mathcal{R}\}$, e.g., $\mathcal{A}_{\mathcal{NC}} = \mathcal{A}_{\mathcal{N}} \cup \mathcal{A}_{\mathcal{C}}$ models a network adversary who also controls a subset of clients.

Probability space. Suppose a security parameter λ , we define the probability space $(\Omega, \mathcal{F}_\sigma, \Pr_\lambda)$ where Ω is the set of all execution traces $\mathcal{E}$ generated by a PPT adversary $\mathcal{A}$ interacting with system $\mathcal{S}$. The σ -algebra $\mathcal{F}_\sigma$ is generated by cylinder sets over finite event prefixes. All probabilities are taken over the joint randomness of $\mathcal{A}$, the blockchain's block-production process, and network scheduling.

2.4 Security Properties

We then define the security goals. A secure x402 system satisfies the following properties, each defined relative to an adversary $\mathcal{A}$ generating executions $\mathcal{E} \leftarrow \mathcal{A}(\mathcal{S})$.

Definition 2 (Authorization Soundness). An x402 system achieves *authorization soundness* if, for every PPT adversary $\mathcal{A}$, the probability that a server R grants access to a client C for resource res at time t without eventual final on-chain settlement is negligible. Let

$$E_{\text{auth}} = \{\exists t, C, R, res, PP : R.\text{grant}(C, res, t) \wedge \neg \text{Final}(tx_{pp}, k)\}.$$

Authorization soundness holds if $\Pr_{\mathcal{E} \leftarrow \mathcal{A}}[E_{\text{auth}}] \leq \text{negl}(\lambda)$.

Definition 3 (Payment–Service Correspondence). Let $\text{GrantCount}(C, res, \mathcal{E})$ denote the number of distinct grants for (C, res) , and $\text{Settlements}(PP, \mathcal{E})$ the number of on-chain settlements for the same logical payment identifier. The consistency failure event is

$$E_{\text{dc}} = \{\exists C, res, PP : \text{GrantCount}(C, res, \mathcal{E}) \neq \text{Settlements}(PP, \mathcal{E})\}.$$

This captures two failure modes:

- grants exceeding settlements (resource server delivers unpaid content, Attack II);
- settlement without service (payer charged but denied access, Attack I-B).

Payment–service correspondence holds if $\Pr_{\mathcal{E} \leftarrow \mathcal{A}}[E_{\text{dc}}] \leq \text{negl}(\lambda)$.

Definition 4 (Replay Resistance). Let PP be a valid PaymentPayload first used at time t . The replay event is

$$E_{\text{replay}} = \{\exists t' > t : R.\text{grant}(C, res, t') \wedge \text{same } PP \text{ reused}\}.$$

The system is *replay-resistant* if $\Pr_{\mathcal{E} \leftarrow \mathcal{A}}[E_{\text{replay}}] \leq \text{negl}(\lambda)$.

Definition 5 (Facilitator k -Atomicity). Let E_{atom} be the event that a facilitator reports a transaction tx as final before k confirmations:

$$E_{\text{atom}} = \{\exists tx, t : \text{Confirmations}(tx, t) < k \wedge F.\text{reportFinal}(tx, t)\}.$$

A facilitator is k -atomic if $\Pr_{\mathcal{E} \leftarrow \mathcal{A}}[E_{\text{atom}}] \leq \text{negl}(\lambda)$.

2.5 Security Theorems

We now state the main consequences of the model above. The theorems formalize when these security properties hold or fail as a function of execution policy, facilitator honesty, and chain finality. We only show key statements; full proofs are placed at Appendix A.

THEOREM 6 (AUTHORIZATION SOUNDNESS—CONSERVATIVE EXECUTION). *Let $\mathcal{S}$ be an x402 system under conservative execution, where R grants access only upon receiving F 's report that $\text{Confirmations}(tx_{pp}, t) \geq k$. Assume (i) F is honest (i.e., $\mathcal{F} \notin \mathbb{A}$), (ii) Assumption 2.3 holds, and (iii) the blockchain satisfies the k -deep finality property (Definition 1). Then for every PPT adversary $\mathcal{A} = \mathcal{A}_N \cup \mathcal{A}_C \cup \mathcal{A}_B$, we have $\Pr_{\mathcal{E} \leftarrow \mathcal{A}}[E_{\text{auth}}] \leq \epsilon_{\text{chain}}(k)$. If additionally $\epsilon_{\text{chain}}(k) = e^{-\alpha k}$ for constant $\alpha > 0$ (exponential finality decay), then choosing $k = \lceil \alpha^{-1}(\lambda + 1) \ln 2 \rceil$ yields $\Pr[E_{\text{auth}}] \leq 2^{-\lambda} = \text{negl}(\lambda)$.*

THEOREM 7 (AUTHORIZATION SOUNDNESS VIOLATION—OPTIMISTIC EXECUTION). *Consider an x402 system under optimistic execution, where R grants access upon F 's verification response before on-chain finality. For any adversary with capability $\mathbb{A}_I = \{\mathcal{N}, \mathcal{F}, \mathcal{B}\}$, let $\Delta = T_{\text{verify}} + \delta$ denote the time from transaction broadcast to service grant, where $\delta \geq 0$ is adversarial delay on the $F \rightarrow R$ path, and let T_{inc} be the inclusion time of tx_{pp} . If pre-finality reorganizations occur with probability at least $p_{\text{reorg}} > 0$ and are independent of HTTP events (Assumption 2.3), then $\text{RGP}_k \geq p_{\text{reorg}} \cdot \Pr[T_{\text{inc}} + k \cdot T_b > \Delta]$. In particular, when $k = 0$, $\text{RGP}_0 \geq p_{\text{reorg}} \cdot \Pr[T_{\text{inc}} > \Delta]$.*

THEOREM 8 (REPLAY RESISTANCE UNDER PRE-GRANT UNIQUENESS ENFORCEMENT). *Let $\mathcal{S}$ be an x402 system in which R atomically claims each $(\text{pay_id}, \text{resource_id})$ before granting service, retains that claim for TTL, and rejects payloads outside a freshness window $[t - W, t + W]$ with $W < \text{TTL}$. Then for every PPT adversary $\mathcal{A} = \mathcal{A}_N \cup \mathcal{A}_C$, we have $\forall t' \in (t, t + \text{TTL}) : \Pr[E_{\text{replay}}] = 0$. Conversely, absent such a pre-grant claim, there exist deployments in which $\mathcal{A}_N$ replaying PP n times achieves $\text{GrantCount}(\text{pay_id}, \mathcal{E}) = n$ with probability 1.*

THEOREM 9 (FACILITATOR k -ATOMICITY AS PREREQUISITE). *If F is not k -atomic, i.e., $\Pr[E_{\text{atom}}] = \mu$ for some non-negligible μ , then under Assumption 2.3, even conservative execution cannot achieve authorization soundness: $\Pr[E_{\text{auth}}] \geq \mu \cdot p_{\text{reorg}}$.*

COROLLARY 10 (CONFIRMATION DEPTH RECOMMENDATION). *For a target authorization-soundness failure probability ϵ_{target} , under conservative execution with an honest facilitator, the minimum confirmation depth is $k^* = \min\{k : \epsilon_{\text{chain}}(k) \leq \epsilon_{\text{target}}\}$. For Base (L2 on Ethereum) with average block time $T_b \approx 2$ s and $\epsilon_{\text{chain}}(k) \approx e^{-\alpha k}$ for chain-specific $\alpha > 0$, we recommend $k \geq 3$ for resources valued below \$1 (achieving $\epsilon_{\text{target}} < 10^{-2}$) and $k \geq 12$ for resources above \$10 (achieving $\epsilon_{\text{target}} < 10^{-4}$).*

These results clarify how the properties relate. Authorization soundness (Definition 2) is violated by Attack I when optimistic execution meets reorgs or Byzantine facilitators (Theorem 7). Payment–service correspondence (Definition 3) is violated by Attack I-B (payment without service) and Attack II (grants without payment). Replay resistance fails under Attack II when idempotency is absent (Theorem 8). Facilitator k -atomicity is a *prerequisite*: if F reports finality before k confirmations, even conservative servers can be deceived (Theorem 9). Since our threat model allows Byzantine facilitators, resource servers that independently verify on-chain finality achieve authorization soundness with respect to $\mathcal{A}_{\mathcal{F}}$.

In the following sections, we instantiate these results through our attacks. We turn the formal failure events into concrete attacks in §3; we then measure the corresponding observables in §4; and we examine whether deployed implementations enforce the theorem preconditions in §4.6.

3 Our Attacks

We now instantiate the security properties from §2.5. Each attack identifies a concrete way to realize one of the formal violation events. Attack IV is the one exception in scope: it occurs before the payment protocol begins (i.e., the discovery layer) and complements the settlement and replay theorems by showing how an agent can be steered toward a bad x402 endpoint in the first place.

3.1 Attack I: Settlement-Path Inconsistencies

x402 decouples *verification* from *settlement*. In the honest path, Client C requests a paid resource from Resource Server R ; R asks Facilitator F whether the payment looks acceptable; and F later settles against Blockchain B . Attack I exploits the gap between the HTTP decision at R and the eventual on-chain state at B .

Attack I is the operational form of the authorization and atomicity results. Attack I-A instantiates Theorem 7: a network- or chain-side adversary widens the gap between “grant” and finality, so the server grants before the payment basis is final. Attack I-B targets a different failure mode: a request-path observer reads the payment header once, races the legitimate facilitator, and consumes a valid authorization without producing the service grant, violating payment-service correspondence.

3.1.1 Attack I-A: Revert-grant under optimistic execution.

Attack intuition. The participants are: Client C (the payer), the resource server R (the party serving resources), Facilitator F (the verifier and submitter on the settlement path), and Blockchain B (the final source of truth). The honest order is simple: C asks for resources, R checks with F , and only then should R release resources.

The vulnerability appears when R optimistically treats a non-final signal as if it were durable payment. The server may see mempool presence, shallow inclusion, or a proxy-level “ok” from F and interpret that as finality, even though the payment can still be replaced away. So the attacker in Attack I-A is not a malformed client input; it is any network or chain actor that can make the gap between R ’s grant and B ’s finality wide enough to matter.

The attacker may delay or reorder the network path ($\mathcal{A}_N$), spoof or corrupt the facilitator ($\mathcal{A}_F$), or influence inclusion, replacement, and bounded reorganization on the chain ($\mathcal{A}_B$). In short, the attacker needs a way to make the gap between “verification looks fine” and “settlement is final” large enough to exploit.

Failure condition. A grant event E_{revert} is the moment when R releases the paid resource to C . I-A succeeds when R grants before the payment reaches the confirmation threshold, and that same payment later disappears from the canonical chain.

Formally, the event is:

$$E_{\text{revert}} = \{g \mid \neg \exists tx_{pp} : \text{Final}(tx_{pp}, k) \text{ at } t_g \\ \vee tx_{pp} \text{ is later absent from the canonical chain}\}.$$

Attack procedure. We now walk through the failure in order.

- **Step 1: Client prepares payment.** Client C requests the resource from Server R . The server responds with 402 and a payment request, after which C sends back the payment header X-PAYMENT.
- **Step 2: Server asks for a quick verdict.** Server R forwards the payment header to Facilitator F through /verify. At this point, F may only be checking that the payment is visible in the mempool or shallowly included.
- **Step 3: Server grants early.** If F returns ok, Server R releases the resource **before**¹ the intended k -confirmations. So the key point is: the resource is already gone, but the payment is still reversible.
- **Step 4: Adversary exploits the gap.** The attacker then uses the remaining window to prevent durable settlement. Depending

¹We use red to mark attack/failure points throughout this section.

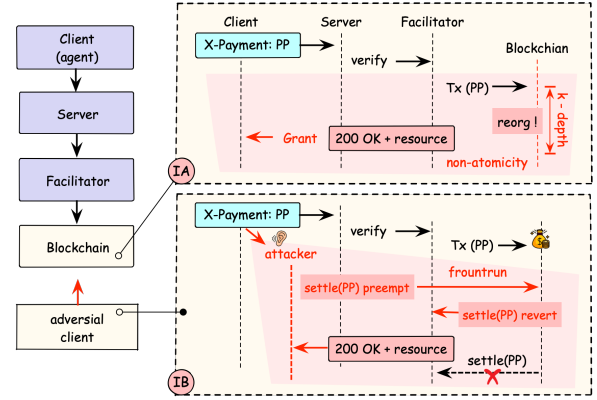


Figure 2: Illustration of Attack I. In Attack I-A, the server grants resources before the payment is finalised onchain, so a later reorg may remove the payment. In Attack I-B, an attacker preempts settle(PP) before the facilitator, so the payment succeeds but the client receives no service.

on its position, it can delay the verification path, replace tx_{pp} via RBF, or trigger a bounded reorganization that removes the transaction from the canonical chain.

- **Step 5: Server cannot roll the grant back.** After the reorg or replacement, the payment no longer exists on the canonical chain. But the HTTP response has already been sent, so R cannot claw the resource back from Client C .

Why the attack works. Attack I-A is a classic *time-of-check vs. time-of-use* problem: R checks that the payment *looks* acceptable but commits the stronger action of releasing the resource as if settlement were final, and the two moments are not atomic. The pattern mirrors blockchain’s “0-conf acceptance” risk—a merchant accepts before sufficient confirmations, and a later double-spend or reorg invalidates that acceptance. The key asymmetry is that B can roll back a pending transfer, but cannot roll back a delivered HTTP response. The failure is most plausible when an honest facilitator only checks mempool visibility or shallow inclusion while the server treats that reply as final. The facilitator need not be Byzantine; a latency-optimized honest deployment creates the same exposure.

3.1.2 Attack I-B: Unauthorized settlement preemption.

Attack intuition. Participants (C, R, F, B) match I-A; what changes is the adversary, a request-path observer or Byzantine server that reads the payment proof before the honest facilitator uses it. The failure is a stolen authorization: the payment is valid but consumed by the wrong caller, so C is charged while R fails to deliver. The attacker can be a network observer (N), e.g., a TLS-terminating proxy or middleware seeing X-PAYMENT in transit, or a Byzantine server (R^*) that receives the header and uses it for its own settlement.

This branch uses the same payment object but flips the failure direction: rather than service without durable payment, it produces durable payment without service, the second half of Definition 3.

Failure condition. Attack I-B succeeds when a payment is consumed on chain, but Server R never produces the corresponding service grant. We denote this by E_{preempt} .

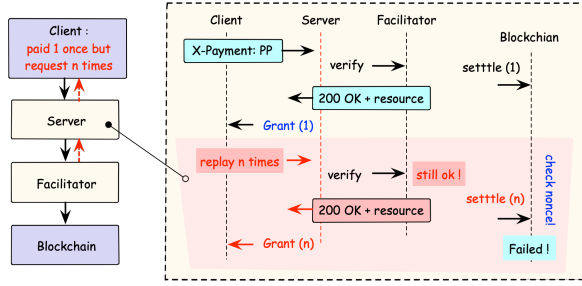


Figure 3: Illustration of Attack II. A client pays once but replays the same X-PAYMENT payload n times. Without a pre-grant idempotency check, the server may issue n resource grants, while the blockchain accepts only one settlement.

The event is the mirror image of replay: replay creates too many grants for one payment, while preemption creates a payment without the expected grant. Formally, the event E_{preempt} is:

$$E_{\text{preempt}} = \{\exists PP : \text{Settlements.}(PP) = 1 \wedge \text{GrantCount}(PP) = 0\}$$

Attack procedure. Here the failure is shorter and more familiar than I-A: it is a race for who gets to use the authorization first.

- **Step 1: Client exposes the payment proof.** Client C sends X-PAYMENT toward Server R .
- **Step 2: Attacker copies the authorisation.** A request-path observer, or a Byzantine server itself, extracts the embedded authorization from the header; the key is **copying a usable payment capability**.
- **Step 3: Attacker races settlement.** Before Facilitator F submits the honest settlement, the attacker submits the same authorization from an attacker-controlled account, i.e., **settles first**.
- **Step 4: Honest settlement fails.** Once the nonce is **consumed**, Facilitator F no longer has a usable path to settle the same authorization. Server R then fails the request or returns 402, even though Client C has effectively already paid.

Why the attack works. The settlement path does not bind the facilitator identity to authorization. In EIP-3009, any observer with the signed authorization can submit the transfer first. In Permit2, a contract without a facilitator field or caller check allows the same preemption. The attacker consumes the nonce, causing the legitimate facilitator’s later settlement to fail despite a valid payment intent. Thus, x402 inherits a frontrunning risk whenever payment proofs are exposed and settlement is not bound to the caller.

Remark. The risk is especially realistic when payment headers cross logging middleware, TLS terminators, or API gateways, which are components never meant to hold spendable payment material. With caller-unbound settlement, any such observer becomes a potential preemption point.

3.2 Attack II: Replay / Idempotency across the HTTP-Chain Boundary

Attack II involves the same four actors as Attack I: Client C , Server R , Facilitator F , and Blockchain B . Its adversary, however, is usually less dramatic: a replaying client, a retry-happy intermediary, or simply a deployment that fails to record that a payment identity has already been used.

Attack II instantiates the converse direction of Theorem 8: unless the server atomically claims $(pay_id, resource_id)$ before granting service, replay becomes a direct payment-service failure, allowing one logical payment to produce multiple HTTP grants.

Attack intuition. In the honest flow, one intended payment buys one protected response; Attack II breaks that one-to-one mapping. x402 carries a bearer-style payment capability in X-PAYMENT, so if neither R nor F maintains an idempotency record keyed to the logical payment identity, the same capability can be returned through retries, concurrent requests, or semantically equivalent encodings and still appear fresh.

The key asymmetry is that B may enforce single-use settlement via a nonce or authorization-used bit, while the HTTP side continues to emit grants. The failure is not “multiple on-chain payments” but “multiple off-chain grants for one intended payment”, which is an ordinary web idempotency bug made expensive by attached money. The attacker capability is correspondingly mundane: resend the same payment, or resend it in forms the HTTP layer treats as fresh, until the server grants more than once.

Failure condition. A *duplicate-grant* event E_{dup} occurs when one logical payment identity leads to more HTTP-layer grants than the application intended. The payment was meant to buy one release of resources, but the system releases it multiple times.

In practice, the event is often triggered by ordinary engineering behavior rather than an obviously malicious client: SDK retries, load-balancer retries, and concurrent workers can all replay the same payment if the idempotency store is missing or scoped to the wrong component. Formally, the event is:

$$E_{\text{dup}} = \{pay_id \mid \text{GrantCount}(pay_id, \mathcal{E}) > \text{IntendedPayments}(pay_id, \mathcal{E})\}.$$

Attack procedure. The failure builds up across layers.

- **Step 1: Client obtains a usable payment header.** Client C completes the usual 402 challenge and prepares a valid X-PAYMENT value for some payment identity pay_id .
- **Step 2: Adversary resends the same payment.** The attacker **reuses the same X-PAYMENT** across retries, concurrent requests, or slightly different HTTP encodings that still refer to the same logical payment.
- **Step 3: Server checks each replay independently.** Server R forwards each replay to Facilitator F and, without a deduplication record, treats every arrival as a fresh authorization attempt.
- **Step 4: The HTTP layer grants more than once.** Server R therefore releases the resource **again for the same** logical payment, even though the application intended only one grant.
- **Step 5: The chain cannot repair the extra grants.** B may still allow **at most one successful settlement**, but that invariant arrives too late: the extra HTTP responses have already been delivered.

Why the attack works. Attack II is a classic idempotency failure. Web systems already know that payment endpoints, callback handlers, and retryable POST operations need exactly-once protection. x402 inherits the same problem, except the mistake now sits between a bearer payment header and an asynchronous on-chain settlement path. The chain’s nonce check is not enough, because

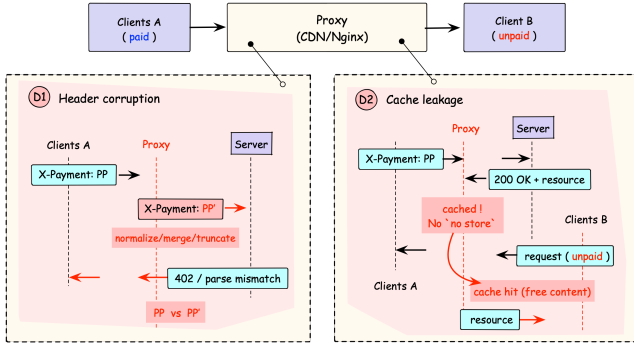


Figure 4: Illustration of Attack III. A client request passes through an HTTP proxy before reaching the resource server. In D1, the proxy rewrites X-PAYMENT, causing a payload mismatch. In D2, the proxy caches a paid response without no-store, allowing a later unpaid client to receive it for free.

it protects only contract state. It does not tell Server *R* whether a prior HTTP request with the same logical payment already caused a resource grant. That is why the attack survives even when the contract itself behaves correctly: the contract enforces “one settlement,” while the web service still fails to enforce “one grant.”

There is also a more subtle version of the same bug: two payloads may be semantically identical while differing in serialization details such as JSON ordering, whitespace, or encoding. If a deployment hashes the raw header bytes instead of a canonical payment identity, replay can hide inside representation changes rather than byte-for-byte duplicates. So the attack is not only about obvious duplicates; it is also about the gap between semantic equality and what the implementation happens to remember.

3.3 Attack III: HTTP / Proxy-Level Confusion and Header Manipulation

Attack III moves one layer earlier, to the HTTP path itself. The key participants are Client *C*, an intermediary *P* such as a reverse proxy or CDN edge, and Server *R*. The adversary is not a chain manipulator, but an intermediary or middleware component that can normalize, rewrite, merge, cache, or log traffic before the origin server observes it. Attack III carries the payment-service property into the surrounding HTTP infrastructure: even when on-chain settlement is correct, proxy rewriting or shared caching can make the service boundary diverge from the payment boundary.

Unlike Attack I and Attack II, the core problem here is not whether the payment is valid on chain. The problem is that x402 places spendable payment material inside ordinary HTTP machinery. Once that happens, the payment header and the paid response inherit the same parsing ambiguities and caching hazards that already plague authenticated web traffic. The two most important failures are different enough that it is better to separate them: (D1) one changes the header in flight, and (D2) the other leaves the payment alone but leaks the paid response later.

Attack intuition (Fig.4). The participants are paying Client *C*, intermediary *P*, Server *R*, and sometimes a later unpaid client *C'*. Attack III is still one attack class: an HTTP intermediary distorts the

payment boundary before the origin has a clean chance to enforce it. In one manifestation, Proxy *P* changes the X-PAYMENT header in flight, so Server *R* reasons about different payment semantics than Client *C* intended. In the other, the request is handled correctly, but the paid response is later replayed because an intermediary caches it as if it were ordinary public content.

These two outcomes have a different flavor. Header corruption (D1) is a parser-risk that depends on how multiple components normalize and interpret the same request. Cache leakage (D2) is cleaner and more concrete: the payment may be perfectly valid, yet the paid artifact later reaches someone who never paid. They belong together because both arise from the same design move, namely carrying spendable payment material and paid artifacts through ordinary HTTP infrastructure.

In our evaluation, cache leakage is the primary validated outcome, while header ambiguity is better understood as a realistic parser-risk that appears once real proxies, CDNs, API gateways, or observability layers are inserted in front of the origin.

Failure condition. Attack III succeeds when an intermediary either causes Server *R* to see payment semantics different from what *C* sent, or replays a paid response outside the original paid context so that an unpaid client receives it.

Formally, we capture the attack as the union of two events:

$$E_{III} = \{req \mid Sem_R(X-PAYMENT) \neq Sem_C(X-PAYMENT)\} \cup \{req' \mid ServedFromCache(req') \wedge \neg Paid(req')\}.$$

Attack procedure. The exact path depends on the intermediary, but the overall failure is simple.

- *Step 1: Client sends a paid request.* Client *C* emits a request carrying X-PAYMENT toward Server *R* through intermediary *P*.
- *Step 2: The intermediary handles payment-sensitive HTTP state as ordinary traffic.* Depending on the deployment, Proxy *P* may **normalize**, **merge**, or **truncate** the payment header, or it may later **cache** the paid response because the application omitted strong directives such as no-store.
- *Step 3: The wrong artifact becomes authoritative.* Server *R* may verify a mutated header *PP'* rather than the client’s original *PP*, or a later unpaid request may be answered from a stored copy of a previously paid response.
- *Step 4: The payment boundary collapses.* The system either acts on a parser view that **changed in transit** or returns paid content to an **unpaid client**.

Why the attack works. Attack III is closest in spirit to classic request-confusion and authenticated-response cache bugs: HTTP components share no canonical interpretation of transformed headers, and shared caches do not natively understand that a response was produced only after a payment gate—they simply follow local semantics. The x402-specific twist is that the ambiguous object is not just an authentication token but spendable payment material or a paid artifact, which raises the stakes of familiar web behaviors such as duplicate-header merging, canonicalization, truncation, logging, and edge caching, all of which become payment-relevant once they sit on the path.

The two manifestations differ: header corruption is a deployment-dependent parser risk where components disagree on the verified payment object, while cache leakage is a cleaner end-to-end exploit—payment settles correctly, yet the resource is later replayed for free because the cache boundary ignored the payment boundary.

3.4 Attack IV: Server-Selection Attacks

Attack IV extends the model upstream of Π_{x402} , from settlement to discovery. While the formal theorems assume a chosen server and analyze whether its payment flow is safe, Attack IV shows that agent-facing discovery can bias which server receives the payment opportunity in the first place. The relevant participants are a discovery layer such as Bazaar, an LLM agent that selects among candidate servers, honest servers R_h , and attacker-controlled servers R^* . The adversary is a malicious seller that need not break cryptography or settlement logic; it only needs to manipulate how the agent perceives, ranks, and selects available servers.

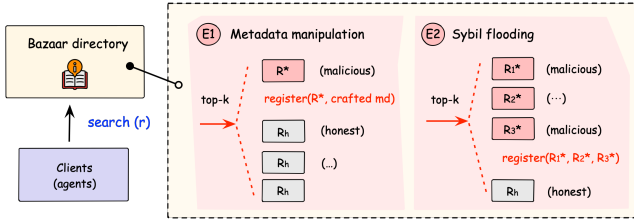


Figure 5: Illustration of Attack IV. The attacker, in E1, crafts metadata to place a malicious server in shortlist; in E2, registers multiple Sybil servers to crowd out honest providers.

Attack intuition (Fig.5). In Bazaar-style discovery, the agent does not manually inspect the whole market. It queries a catalog, receives a shortlist, and then chooses one server based on names, descriptions, prompts, prices, trust cues, or latency cues. Attack IV exploits that decision pipeline. If an attacker can shape the metadata or multiply its presence through Sybil listings, the agent can be nudged toward an attacker-controlled paid endpoint before it ever reasons carefully about the market as a whole.

This matters more in x402 than in an ordinary tool-selection mistake because discovery is directly connected to payment. Once the agent selects a server, that server can immediately issue payment requests and influence the agent’s next action. The attacker does not need to compromise the wallet or the settlement contract; it only needs to win the selection step often enough. The harm is far beyond poor service choice but on-chain economic loss.

Failure condition. A *selection-bias event* occurs when an attacker-controlled server is chosen more often than the baseline market would justify after discovery has been manipulated.

Let $\mathcal{R}_k$ denote the shortlist returned to the agent for a query q , and let $\pi_\Delta(R^* | q, \mathcal{R}_k)$ be the probability that the agent selects the adversarial server under manipulation Δ . We capture the success event as $E_{\text{sel}} = \{q | \pi_\Delta(R^* | q, \mathcal{R}_k) > \pi_0(R^* | q, \mathcal{R}_k)\}$.

Core manipulations. The cleanest forms of Attack IV are the first two. In E1: *metadata manipulation*, the attacker rewrites names,

descriptions, and LLM-facing prompts so that R^* appears more relevant than honest competitors. In E2: *Sybil flooding*, the attacker registers multiple aliases $R_1^*, \dots, R_r^*$ so that adversarial entries occupy more of the shortlist returned by discovery.

The remaining variants are better read as extensions than as the core story: E3 manipulates price and payment signals, E4 leans on latency and availability cues, and E5 exploits how facilitator affiliation or payment-path metadata is presented to the agent. They matter, but they all build on the same underlying move: shape the shortlist first, then let the agent do the rest.

Among the five variants, E1 and E2 are the cleanest core attacks as they require only ordinary seller-side registration powers. E3–E5 are best read as extensions that manipulate richer marketplace signals after the attacker has already gained a place in the shortlist.

Attack procedure. The manipulation unfolds before the victim ever sends a payment.

- *Step 1: Adversary prepares the listing.* The attacker **registers crafted metadata** for one malicious server or several Sybil aliases.
- *Step 2: Discovery returns a shortlist.* For a target query q , the discovery service returns top- k candidate servers to the agent.
- *Step 3: The shortlist is biased.* Since the adversary has tuned relevance, quantity, or trust cues, the malicious entries occupy **more of the visible search surface** than under a clean baseline.
- *Step 4: The agent picks the wrong server.* The agent then selects **the attacker-controlled server R^*** and routes traffic for payments.
- *Step 5: Economic harm follows selection.* Once discovery has already steered the workflow toward R^* , the payment path and service interaction proceed on top of a manipulated choice.

Why the attack works. Attack IV resembles tool-selection and ranking attacks on LLM agents: discovery compresses the catalog into a shortlist, then the model decides from that filtered view. Once the attacker enters the shortlist with persuasive metadata, the decision is often lost before deeper reasoning begins.

The x402 setting adds two amplifiers. First, Bazaar-like ecosystems often have weak registration controls, so adversaries can cheaply create or rename listings until one lands in the right lexical neighborhood. Second, the chosen server is monetized, so a bad selection translates directly into payment requests, poor service, or financial loss. Sybils compound this by raising the chance that at least one adversarial listing wins. Attack IV is thus a market-surface attack: the adversary need not outperform every honest server, only be more visible or persuasive when the shortlist is formed.

4 Evaluation

This section shows our experimental design and results. We first summarize the shared configuration settings that apply across the evaluation. We then evaluate each attack class in turn.

The evaluation follows the violation events defined in §2.4. RGP_k measures authorization-soundness failures (Definition 2) under optimistic or non-atomic settlement; DGR measures replay payment-service failures (Definition 3 and Theorem 8); cache leakage captures violations at the HTTP boundary (Definition 3); and selection rate captures the discovery-layer precursor that determines which x402 server receives the payment flow (§3.4).

4.1 Shared Configuration Settings

System components. Our testbed consists of four components:

- **Resource Server R** (Node.js 20 / Express): implements the full x402 payment flow, including the initial 402 challenge, X-PAYMENT receipt, /verify call, and access decision. We configure R in an optimistic mode (grant soon after settlement submission) or a conservative mode (wait for explicit confirmation depth k).
- **Facilitator F** (Node.js / Express / ethers.js v6): verifies signatures and settles, supporting *honest* (wait for receipt), *optimistic-bug* (reply “ok” on mempool or shallow inclusion), and *byzantine* (return “ok” without on-chain enforcement) modes.
- **Blockchain B** : a local Hardhat chain for controlled reorg simulation and Base Sepolia for realistic latency and live endpoint validation for attack surfaces.
- **Client harness C** : issues concurrent x402 flows, replays headers, and drives proxy and discovery experiments.

Hardware configurations. All local experiments run on a single Apple M3 Max workstation with a 16-core CPU under macOS.

Instrumentation. R records per-grant metadata (*pay_id*, tx hash, grant time, policy, block number); F logs verify calls, settlement attempts, duplicates, and reverts. Outputs are serialized to times-tamped JSON traces for reproducibility.

Reorg and delay injection. On the local chain, a reorg injector snapshots state, mines a branch containing tx_{pp} , and reverts with probability p_{reorg} . For larger sweeps, we use an analytic model with a Bernoulli revert trial with probability p_{reorg} at $k = 0$ and p_{reorg}^k for $k > 0$. Application-level delay $\delta \in \{0, 100, 200, 400\}$ ms is injected on the $R \rightarrow F$ path. Attack I-B is evaluated separately via Base Sepolia preemption traces. For Attacks II and III, the harness replays headers under controlled concurrency through intermediaries that may mutate or cache content.

Parameter space. Attacks I-A/B vary $k \in \{0, 3, 6, 12\}$, $p_{\text{reorg}} \in \{0, 0.01, 0.05\}$, $\delta \in \{0, 100, 200, 400\}$ ms, facilitator honesty, and execution policy. Attack II varies replay count $n \in \{1, 5, 10, 50\}$ and idempotency keying. Attack III runs $\sim 1,000$ requests per condition through nginx, Caddy, and a custom MitM proxy. Attack IV evaluates 12 categories $\times$ 15 queries across three LLMs with Sybil count $r \in \{1, 3, 5\}$. Local state is reset between configurations.

External systems and artifacts. To avoid overfitting to a purely local stack, we also evaluate four public x402-enabled testnet endpoints across different application categories: blockchain data, general infrastructure, search/analytics, and domain-specific APIs. They are anonymized in the submission version for coordinated-disclosure reasons. We also audit additional x402 SDKs in TypeScript, Python, and Rust to compare replay handling, settlement binding, and header processing across codebases.

Smart contract and statistics. Our controlled on-chain experiments use a MockUSDC contract that faithfully implements EIP-3009 transferWithAuthorization, including per-authorizer nonce tracking, EIP-712 signing, and authorization-used events. For rate metrics such as RGP_k , DGR, cache leakage incidence, and selection rate, we report point estimates with 95% Wilson score confidence intervals using $z = 1.96$. Latency metrics such as T_{gf} are summarized using the median and interquartile range (IQR).

4.2 Evaluating Attacks I-A and I-B

Experimental setup. For Attack I-A, we sweep confirmation depth $k \in \{0, 3, 6, 12\}$, reorg probability $p_{\text{reorg}} \in \{0, 0.01, 0.05\}$, and delay $\delta \in \{0, 100, 200, 400\}$ ms on the local chain, comparing optimistic vs. conservative execution and honest vs. Byzantine facilitators. Each run uses a fresh EIP-3009 authorization with the chain reset between configurations, and we retain one end-to-end trace illustrating the revert-grant path. Base Sepolia serves as a timing sanity check for the grant-to-finality gap but is not used to estimate revert probability, since public-testnet reorgs cannot be injected.

For Attack I-B, we validate two settlement paths on Base Sepolia: EIP-3009 preemption against a live endpoint settling via transferWithAuthorization, and a Permit2 PoC against the x402ExactPermit2Proxy contract. In both, the adversary observes X-PAYMENT before the honest facilitator settles.

Metrics. For Attack I-A we measure RGP_k , the probability of a revert-grant event at depth k , and T_{gf} , the time gap between the HTTP grant and durable chain finality. For Attack I-B we record whether an attacker can consume the authorization before the expected facilitator path settles, yielding payment without service. This is intentionally a binary validation metric rather than a broad rate estimate: the point of Attack I-B is to show that the settlement path is caller-unbound and therefore preemptable at all.

Results on Attack I-A (Fig. 6). Attack I-A shows a security-latency tradeoff. In the local sweep, revert-grant events occur when the server grants on a non-final signal and reorg probability is non-negligible. At $k = 0$ in the fixed $T_b = 2$ s main sweep, RGP_0 reaches 4.70% under $p_{\text{reorg}} = 0.05$ and $\delta = 200$ ms, and rises to 5.18% when the injected delay increases to 400 ms. Increasing k reduces observed risk to the CI floor, but raises T_{gf} to tens of seconds. Stronger confirmation mitigates Attack I-A at the cost of user-visible latency.

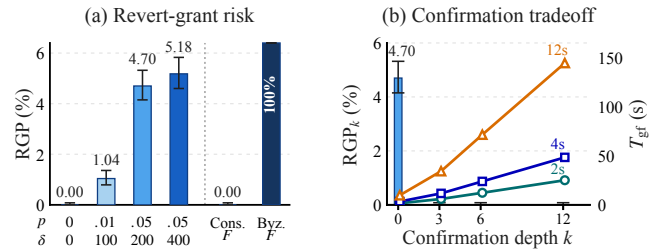


Figure 6: Attack I-A results with $T_b=2$ s and 5,000 requests per condition. (a) RGP_0 grows with reorg probability and delay. (b) Increasing k lowers RGP_k to the CI floor but raises T_{gf} from 1.6 s to 25.1 s.

Control cases bound the behavior: conservative execution with an honest facilitator yields no revert-grants, while a Byzantine facilitator reaches the upper bound. The local trace reproduces the failure path: C pays, F approves, R grants before finality, and an analytic reorg removes tx_{pp} . Base Sepolia confirms that the grant-to-finality gap exists outside the lab, but cannot estimate revert probabilities because public-testnet reorgs cannot be injected.

Results on Attack I-B. Attack I-B is a feasibility validation rather than a rate estimate. The live EIP-3009 trace exercises the full HTTP

path on an anonymized Base Sepolia endpoint. After the payer’s X-PAYMENT header is exposed, the same authorization is submitted directly to testnet USDC through `transferWithAuthorization`; the on-chain transfer succeeds in tx `0x32fb...e556d`, consumes the nonce, and charges the payer 0.0001 testnet USDC. The endpoint’s later settlement attempt then fails on the consumed nonce, and the resource request returns HTTP 402.

We observe the same caller-unbound pattern in a controlled Permit2 proof of concept against `x402ExactPermit2Proxy` on Base Sepolia. An unrelated attacker EOA calls `settle()` with the observed Permit2 signature, and the settlement succeeds in tx `0x3f30...93f1`, consuming the nonce. A later legitimate replay reverts because the nonce has already been used. Together, the EIP-3009 and Permit2 traces isolate the same preemption condition: an attacker consumes the payment capability before the expected settlement path can use it. The EIP-3009 trace shows the paid-but-denied HTTP outcome, while the Permit2 trace confirms the same nonce-consumption failure at the settlement contract.

Takeaways. `x402` safety depends on deployment choices: it requires either extra latency or stricter settlement binding.

4.3 Evaluating Attack II

Experimental setup. We replay the same payment payload concurrently with replay count $n \in \{1, 5, 10, 50\}$. Each request carries the same logical payment identity: identical `pay_id`, nonce, signature, and base64-encoded X-PAYMENT header. The local trials use real settlement on MockUSDC through `transferWithAuthorization`, so each authorization can settle at most once. We run the same matrix without idempotency and with idempotency keyed by `pay_id`.

We then test a live Base Sepolia endpoint, Endpoint-2, using a barrier-released batch of 1,000 concurrent replays of one payment header. This deployment uses PAYMENT-SIGNATURE, an X-PAYMENT-equivalent payment header. We inspect Python and Rust SDKs to identify implementation choices that may widen the replay window.

Metrics. The primary metric is DGR, the number of HTTP grants per intended payment identity. For live testnet replay, we record the on-chain settlement count to separate duplicate grants from duplicate payments. Grants are counted as HTTP 200 responses. Settlement count is estimated from USDC balance delta divided by the per-request price (\$0.001). Response hashes and balance deltas are logged to distinguish HTTP grants from on-chain settlements.

Results on Attack II (Table 2). The local replay matrix shows the core mismatch: the chain enforces one settlement, but the HTTP layer may grant multiple times. Without `pay_id` deduplication, every replay produces another grant, giving $\text{DGR} = n$. With idempotency at the HTTP boundary, duplicate grants disappear. Thus, on-chain nonce discipline does not by itself provide exactly-once service semantics off-chain.

The SDK evidence shows that implementation choices can widen this gap. SDK-B has grant-before-settle behavior: its streaming path flushes the paid response before `settle_payment` runs, so the client can receive the service even when settlement later fails. SDK-C depends on mode. In optimistic mode, it dispatches settlement via `tokio::spawn` and returns immediately, leaving a replay window in which repeated requests can pass verification and reach $\text{DGR} = n$.

Table 2: Attack II: HTTP grants when replaying the same X-PAYMENT n times. Each cell shows the number of HTTP-layer grants (= DGR) per single intended payment.

Implementation	$n=5$	$n=10$	$n=50$	Idempotency
Testbed (no check)	5	10	50	✗
Testbed (<code>pay_id</code>)	0	0	0	✓
SDK-B (Python)	5	10	50	✗
SDK-C optimistic	5	10	50	✗
SDK-C pessimistic	1	1	1	via settle

In pessimistic mode, settlement occurs synchronously before the response, so duplicate nonces revert and DGR drops to 1.

The live testnet replay confirms the duplicate-grant effect under a deployed endpoint. On Endpoint-2, we replay one payment header across 1,000 concurrent requests and measure both grants and USDC settlements. In the strongest positive round, we observe 248 HTTP-layer grants and 1 on-chain settlement. This establishes duplicate granting at the HTTP layer while the on-chain payment is consumed only once.

Takeaways. Attack II does not break settlement: the on-chain authorization is consumed only once. The failure is at the HTTP layer: if the server does not remember used payments (i.e., idempotency), the same payment can trigger multiple grants.

4.4 Evaluating Attack III

Experimental setup. We insert intermediary layers between the client and R to test header mutation and cache leakage separately. The local harness uses an Express origin that logs headers and mimics paid 200 responses with PAYMENT-RESPONSE but no default Cache-Control. We test nginx with `proxy_cache`, default Caddy, and a custom MitM proxy that injects duplicate payment headers. Header tests use unique URLs to force origin hits. Cache tests reuse one URL and restart proxies between conditions for a cold cache.

We also audit public `x402` testnet endpoints settling via Base Sepolia USDC: we record the Cache-Control on a paid response, replay the payment signature five times, and issue an unpaid request for the same resource.

Metrics. We measure whether the payment header changes in transit (Header Mutation Rate) and whether a paid response is later served to an unpaid client (Cache Leakage Incidence). For duplicate-header injection, we also record how the downstream stack resolves multiple payment-header values, since this captures parser ambiguity more directly than a standalone count.

Table 3: Attack III: proxy and caching behavior across middleware stacks. 1,000 requests per proxy per test.

Proxy	Header Mutation	C. Leak (no CC)	C. Leak (with CC)
nginx (Docker)	0.0%	100.0%	0.0%
Caddy (Docker)	0.0%	0.0%	0.0%
Custom MitM	100.0% (multi-header)	—	—

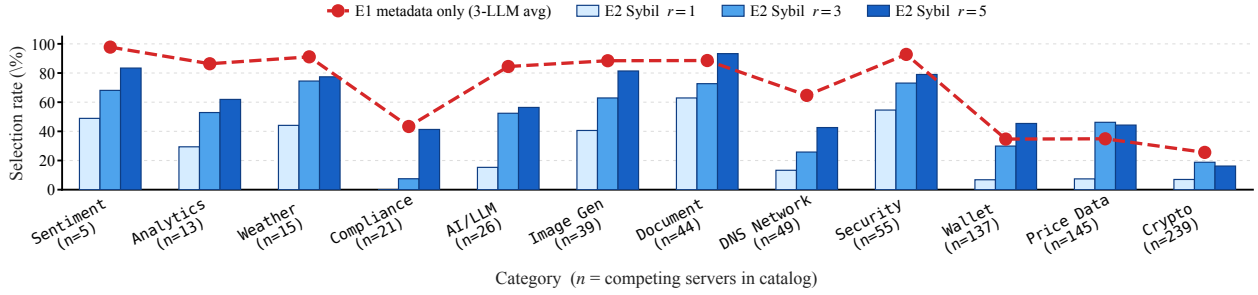


Figure 7: Attack IV selection rates across 12 categories ordered by competition density. Bars: E2 Sybil flooding at $r \in \{1, 3, 5\}$ (3-LLM avg). Red dashed line: E1 metadata-only baseline. n = competing servers per category.

Results on Attack III (Table 3). The local proxy tests show that the two forms of Attack III differ in stability. Header corruption is possible but configuration-dependent. Neither nginx nor Caddy rewrites X-PAYMENT in our setup (0% mutation across 2,000 requests), while a custom MitM proxy injects a duplicate X-PAYMENT in 100% of 1,000 requests, with the Node/Express stack consistently exposing the last value. This establishes parser ambiguity but does not itself constitute an end-to-end bypass.

Cache leakage is direct. With nginx proxy_cache enabled and no Cache-Control protection, all 1,000 unpaid requests receive the cached paid content. When the upstream emits Cache-Control: no-store, private from a cold cache, 0/1,000 unpaid requests are served from cache. Caddy does not cache in either condition.

The live audit shows the same risk. Endpoint-1 returned publicly cacheable paid responses, allowing unpaid clients to retrieve cached content during the CDN cache window. More broadly, audited server-side SDKs do not automatically add Cache-Control: no-store or private to payment-gated responses, leaving cache safety to application or CDN configuration.

Takeaways. The operational risk is not header mutation but cache state escaping the payment boundary. Once paid content is publicly cacheable, later clients can retrieve it without paying.

4.5 Evaluating Attack IV

Experimental setup. We emulate x402_discover using a public x402 endpoint² catalog. The retriever matches queries against endpoint names, descriptions, and tags, uses trust score as a tiebreaker, and returns a top-10 shortlist. This approximates Bazaar-style discovery while letting adversarial entries surface by normal ranking.

We test 12 categories with 15 queries each across three LLMs: MiniMax-M2.7, GPT-5.3, and Sonnet 4.5, all at temperature 0.1. In E1, the attacker adds one crafted server per category. In E2, the attacker adds $r \in \{1, 3, 5\}$ Sybil servers per category, each with a distinct domain and realistic attributes. Overall, we collect 540 LLM decisions for E1 and 1,620 for E2, totaling 2,160 discovery decisions.

Metrics. The primary metric is selection rate, defined as the fraction of valid model decisions for which an attacker-controlled server is chosen. We also track applicability, since selection requires the

malicious listing to appear in the returned shortlist. For E2, we compare aggregate selection rate as the Sybil count increases from $r=1$ to $r=5$.

Results on Attack IV (Fig.7). Discovery can be captured before payment execution begins. The attacker does not need to dominate the full catalog; it only needs to enter the shortlist used by the agent. In E1, a single attacker server with crafted metadata achieves 71.8% selection on MiniMax-M2.7, 69.4% on GPT-5.3, and 68.8% on Sonnet 4.5 across all 12 categories. The similar pattern across models suggests that the effect mainly comes from the discovery setup rather than a single model. Table 4 breaks the E2 Sybil results down by model.

Table 4: Attack IV E2 per-model breakdown: Sybil selection rate (%) by model and Sybil count r .

Category	MiniMax-M2.7			GPT-5.3			Sonnet 4.5		
	$r=1$	$r=3$	$r=5$	$r=1$	$r=3$	$r=5$	$r=1$	$r=3$	$r=5$
sentiment	46.2	75.0	84.6	46.7	60.0	80.0	53.8	69.2	85.7
analytics	30.8	58.3	61.5	26.7	46.7	60.0	30.8	53.8	64.3
weather	46.2	76.9	85.7	40.0	73.3	73.3	46.2	73.3	73.3
compliance	0.0	9.1	57.1	0.0	13.3	46.7	0.0	0.0	20.0
ai_llm	18.2	57.1	69.2	13.3	53.3	53.3	14.3	46.7	46.7
image_gen	50.0	64.3	78.6	33.3	60.0	80.0	38.5	64.3	85.7
document	64.3	71.4	93.3	60.0	73.3	93.3	64.3	73.3	93.3
dns_network	16.7	30.0	41.7	13.3	20.0	40.0	10.0	27.3	46.2
security	57.1	72.7	76.9	60.0	80.0	80.0	46.7	66.7	80.0
wallet	7.1	36.4	42.9	6.7	26.7	46.7	6.7	26.7	46.7
price_data	7.7	55.6	50.0	6.7	40.0	40.0	7.7	42.9	42.9
crypto	7.7	23.1	15.4	6.7	20.0	20.0	6.7	13.3	13.3
Mean	29.3	52.5	63.1	26.1	47.2	59.4	27.1	46.5	58.2

Competition density reduces but does not remove the effect. Sparse categories such as sentiment and weather are highly exposed, while dense categories such as crypto are more resistant. E2 shows the Sybil effect: as the attacker adds realistic aliases, aggregate selection increases from 27.5% at $r=1$ to 60.2% at $r=5$, with document generation reaching 93.3%.

We further compare E2 with a live-registry snapshot collected in April 2026. It contains 13,760 endpoints across 420 domains, with the top nine domains accounting for 87.8% of registrations and one

² <https://x402scout.com/>

domain accounting for 77.5%. We anonymize domains because the relevant issue is concentration rather than attribution.³

Takeaways. x402 can fail before payment begins. If discovery steers the agent to an adversarial shortlist, the payment flow inherits that choice. Metadata validation, ranking controls, and Sybil resistance should be treated as security requirements.

4.6 Cross-Implementation Audit

The audit connects the formal model to implementation practice. Instead of treating SDK issues as isolated bugs, we examine whether real deployments enforce the assumptions required by our theorems: confirmation gating, facilitator-bound settlement, pre-grant uniqueness, resource binding, settle-before-grant ordering, and cache hygiene.

Experimental setup. Instead of reproducing a single exploit trace, we ask whether existing implementations close the underlying gaps. We audit three x402 SDK families: the Coinbase reference TypeScript stack (SDK-A), a third-party Python integration (SDK-B), and a third-party Rust middleware implementation (SDK-C). We also probe four anonymized live testnet endpoints using the same Base Sepolia client as in the earlier experiments.

We cover six properties: confirmation gating before grant, timestamp-window enforcement, *pay_id* idempotency, *resource_id* binding, settlement completeness (verify and settle before grant), and cache-control protection on payment-gated responses.

Results (Table 5). The audit shows that basic checks are more common than boundary-closing ones. All three SDKs enforce timestamp windows, but most of the stronger cross-layer checks are missing. Confirmation gating is shallow: SDK-A and SDK-C’s pessimistic mode gate at $k=1$, while SDK-B and SDK-C’s optimistic mode grant at $k=0$; these defaults do not provide sufficient reorg resistance under the Attack I-A model (§4.2). The same pattern appears in replay and binding logic: none of the audited stacks binds payments to the intended resource, and *pay_id* idempotency is absent except that SDK-C’s pessimistic mode avoids duplicate grants by settling synchronously before responding.

Table 5: SDK audit results.

Check	SDK-A (TS)	SDK-B (Py)	SDK-C (Rust)	Sev.
<i>k</i> -confirmation gating	✗ [†]	✗	✗ [†]	H
Timestamp window	✓	✓	✓	H
<i>pay_id</i> idempotency	✗	✗	✗ [*]	C
<i>resource_id</i> binding	✗	✗	✗	C
Settle before grant	✓ ^c	✗ ^a	~ ^b	C
Cache-Control	✗	✗	✗	M

✓ = enforced; ✗ = missing; ~ = partial. Severity: C = Critical, H = High, M = Medium.

^a SDK-B completes before settlement; later failures are dropped. ^b SDK-C fails only in optimistic mode. ^c SDK-A settles before grant but does not wait long enough for reorg resistance. ^{*} SDK-C fails *pay_id* idempotency only in optimistic mode.

[†] SDK-A/C in pessimistic mode gate at $k=1$; SDK-B/C in optimistic mode gate at $k=0$.

The row-level results show the practical impact. SDK-B violates settle-before-grant because its streaming path finishes before

³The anonymized snapshot is included in our artifact: https://anonymous.4open.science/r/x402-attack-FDF1/results/attack4/cdp_bazaar_catalog_pretty.json.

settle_payment runs, so later settlement failures are dropped. Under nonce pre-consumption, this produces $DGR = n$ without any successful on-chain settlement. SDK-C depends on its mode: pessimistic mode returns HTTP 402 when settle() fails, while optimistic mode sends settlement in a detached tokio::spawn task, allowing the client to receive 200 OK even if settlement later fails.

The cross-implementation view reveals two broader issues. First, no audited SDK binds a payment to the intended resource: a payment signed for resource *A* on Endpoint-1 can be replayed for resources *B*, *C*, and *D* on the same server. Second, deployed settlement paths do not consistently bind msg.sender to the payer-endorsed facilitator. In EIP-3009, any observer can submit the signed transferWithAuthorization payload. In Permit2, x402ExactPermit2Proxy omits caller restriction in settle(), while x402UptoPermit2Proxy binds the facilitator through its Witness and enforces it.

Cache hygiene is also inconsistent. No SDK automatically adds Cache-Control: no-store or private to payment-gated responses, consistent with the leakage observed when Endpoint-1 exposed publicly cacheable Cache-Control. Live endpoints do enforce some simpler checks: malformed headers return HTTP 402, zero or underpaid amounts are rejected, and expired timestamps are blocked by the facilitator’s off-chain validBefore check before any on-chain call.

Takeaways. The audit shows that x402’s risks stem from HTTP-chain boundary rather than isolated SDK bugs. They should be closed at the protocol/implementation levels by default.

4.7 Attack costs

Our attacks do not require large upfront capital. For Attack I-A, we do not price chain reorgs: the relevant failure is that optimistic execution exposes a pre-finality window that can be hit by existing reorg or settlement uncertainty, without any additional x402 payment. Attack I-B only requires an observed payment authorization and a preempting L2 transaction fee, typically at the cent level (e.g., $\sim \$10^{-2}$). Attack II requires one valid x402 payment, but the same payment can yield many HTTP grants if pre-grant idempotency is missing (e.g., one \$0.001 payment produced hundreds of grants in our live test). Attack III needs at most one paid request to create a cacheable response, after which unpaid requests to the same URL can be served from cache (about one service price, e.g., $\sim \$10^{-3}$ in our test). Attack IV has almost no direct on-chain cost because metadata manipulation and Sybil listings only require attacker-controlled endpoint entries, not settlement transactions (protocol-side cost $\approx \$0$, excluding hosting or domain costs).

5 Mitigations

x402 needs tighter constraints at three layers: the payment object, the server grant path, and the deployment environment. We propose six mitigations mapped to these layers (Table 6).

M1–M3 secure the payment capability by binding the resource, the authorized redeemer, and the used-payment state. M4 controls when the service may be released. M5–M6 harden the surrounding HTTP and discovery boundary.

Table 6: Attack-to-mitigation design map.

Attack	Map.	Rule
All	M1	Typed fields and freshness validation.
I-A	M4	Release service only after reserve or k -finality.
I-B	M2	Require caller to match the endorsed facilitator.
II	M3	Bind resource scope and claim the payment once before grant.
III	M5	Use no-store caching and isolate payment headers.
IV	M6	Validate metadata and limit Sybil steering.

M1: Canonical encoding and nonce/timestamp. Payment payloads should use a single canonical typed encoding, such as EIP-712-style structured data, and sign at least `pay_id`, `resource_id`, `facilitator`, `amount`, `token`, `chain_id`, and `expiry` under one typed structure. Servers and facilitators should reject malformed encodings, expired timestamps, duplicate `pay_id` values, and out-of-window nonces before settlement. This reduces cross-SDK serialization drift and makes both `resource_id` scope and `facilitator` identity explicit signed properties rather than deployment-local conventions.

M2: Facilitator-bound settlement. Across settlement paths, the caller with payment capabilities must be the payer-endorsed facilitator. On Permit2 paths, Witness must include a `facilitator` field and `settle()` must enforce `msg.sender == witness.facilitator`; on EIP-3009 paths, settlement should be routed via a facilitator-bound wrapper or equivalent entry point that performs the same check before invoking `transferWithAuthorization`. This closes the preemption path behind Attack I-B. Private relays may reduce mempool-level visibility, but they are only secondary hardening; they do not stop a request-path observer or Byzantine server that has already seen a usable X-PAYMENT header.

M3: Single-use grants and resource binding. Servers should treat X-PAYMENT as a single-use capability and atomically claim the `pay_id-resource_id` pair before releasing any protected response. Resource binding and idempotency are distinct checks: the server must first verify that the signed `resource_id` matches the requested endpoint, then reject duplicate claims for the same pair within a bounded TTL window. This prevents cross-resource first use and replay-driven DGR = n over-granting, including the grant-before-settle behavior observed in audited SDKs. We provide example code below to show the intended order of checks.

```
const payment = parseXPayment(req.header("X-PAYMENT"));
const intent =
  await verifyTypedPayment(payment, requirements);
if (intent.resourceId !== req.path) return reply402(res);
if (intent.expiresAt <= Date.now()) return reply402(res);
const claimKey =
  [intent.payId, intent.resourceId].join(":");
if (!(await claims.claim(claimKey, ttlMs)))
  return rejectReplay(res);
const verdict =
  await facilitator.verify(payment, requirements);
if (!verdict.isValid) return reply402(res);
if (policy === "conservative") {
  await waitForFinality(verdict.txHash, k);
} else {
  await reserve(payment, requirements);
}
res.set("Cache-Control", "no-store, private");
return grantPaidResponse(res);
```

M4: Two-phase settlement. To reduce Attack I-A risk, low-latency deployments should use a reserve-then-settle pattern, or an equivalent two-phase flow in which the server grants only after observing a concrete settlement precondition. Higher-value resources should instead wait for k confirmations according to Corollary 10. The goal is to separate “looks payable” from “safe to release,” so that operators choose the residual revert-grant exposure explicitly rather than inheriting it from facilitators.

M5: Cache and header hygiene. Payment-gated responses must carry `Cache-Control`: no-store or private, and paid routes should bypass intermediary caches where possible. Proxies and CDNs should also avoid normalizing, merging, or duplicating X-PAYMENT values across requests, since cache leakage and parser ambiguity can break payment-service correspondence even when chain settlement is correct.

M6: Agent-selection defenses. For Bazaar-style deployments, mitigation must also cover discovery. Discovery layers should validate metadata, reject unverifiable or prompt-injection-like listing claims, apply Sybil-resistant registration or reputation weighting, and diversify ranking so that no single provider can dominate the shortlist. These defenses do not replace protocol binding, but they reduce the steering surface exposed by Attack IV. Together, M1–M6 make x402’s security boundary explicit: they specify *what* is being paid for, *who* may settle it, and *when* service may be released.

6 Discussion

In this part, we extend our findings.

6.1 Security–Latency Tradeoff

Our results show a security–latency tradeoff in x402 deployment: increasing k reduces measured RGP_k in our local/model-based Attack I evaluation, but increases the grant-to-finality delay, as summarized in Figure 6. In the fixed $T_b=2$ s sweep, median T_{gf} grows from 6.0 s at $k=3$ to 25.1 s at $k=12$, while RGP_k stays at the CI floor. In the overnight $T_b=12$ s sweep, median T_{gf} reaches 34.3 s, 71.5 s, and 144.5 s for $k=3, 6, 12$, respectively. These values should be interpreted as guidance under our controlled reorg model and timing assumptions, not as live-network frequency estimates. In practice, operators should choose k based on resource value, chain characteristics, and acceptable user-visible latency (Corollary 10).

6.2 Threats to Validity

Measurement realism. Our Attack I study mainly uses a controlled local-chain model rather than naturally occurring public-chain reorganizations. In the main sweep, revert-grant risk is estimated using the analytic model $P(\text{revert}) = p_{\text{reorg}}^k$. The Hardhat snapshot/revert injector is used for concrete trace validation, since repeated full-chain rollbacks are too costly at sweep scale. Base Sepolia provides a timing sanity check, but we observed no natural reorgs during the measurement window. Thus, the reported RGP_k values should be interpreted as controlled estimates rather than live-network frequencies.

Coverage. Our implementation coverage is meaningful but not exhaustive. The audit covers three SDK families (TypeScript, Python, and Rust) and four live testnet endpoints, but other stacks may

expose different behaviors. The Attack IV study covers three LLMs, 12 application categories, and 2,160 discovery decisions over an x402scout-based catalog. This is sufficient to show a consistent effect in our setting, but not to claim that all agent architectures, ranking pipelines, or registries behave similarly.

Metric scope. Our metrics target the violation events defined in this paper: RGP_k for revert-grants, DGR for replay-driven over-granting, cache leakage incidence for Attack III, and selection rate for Attack IV. They capture direct protocol and deployment failures, but not downstream effects such as reputation loss, user churn, or failures across larger multi-agent workflows. The DGR-1000 live replay test estimates settlement count from balance deltas and observed block heights; this is practical, but may miss settlements that occur outside the observation window.

6.3 Compatibility with EIPs

We do not evaluate an EIP-native x402 implementation in this paper, but emerging intent standards provide a concrete migration direction for the protocol. EIP-8004⁴ (“Trustless Agents”) points toward typed payment intents rather than deployment-local JSON payloads. For x402, the main value is not the label of the standard by itself but the opportunity to make security-critical bindings explicit and signed under a common schema. An illustrative x402-compatible typed intent could look as follows:

```
{
  facilitator: 0xFac...,
  resource: "/v1/weather?city=Sydney",
  token: USDC, amount: 10000, expiry: 1712345678,
  hook: "x402.verifyAndSettle"
}
```

The highlighted fields show where typed intents help. A signed `facilitator` field provides a binding point for Attack I-B, while a signed `resource` field supports the payment–resource coupling needed for Attack II. An explicit `hook` field, used here as an x402-specific integration hook rather than a claim about EIP-8004’s current field names, makes the verification and settlement entry point visible to wallets, facilitators, and resource servers under the same typed structure. Typed-intent standards, therefore, offer x402 a path toward canonical encoding, stronger cross-implementation binding, and more auditable interoperability, although they do not eliminate timing or caching risks.

At the signing layer, this is consistent with EIP-712, which defines typed structured-data signing, and with ERC-3009, whose `transferWithAuthorization` path already uses typed-signature machinery in our study. ERC-8004 is therefore best viewed not as a payment standard for x402 itself, but as a complementary agent-registry layer that could carry richer trust, discovery, and payment-related metadata around x402-enabled services.

6.4 Design Implications for Protocol Developers

Our findings suggest that x402 implementations should treat payment verification as a cross-layer authorization process, not as a simple HTTP header check. A server should decide how long to wait

before granting access based on the value of the requested resource and the stability of the underlying chain. Low-value API responses may tolerate a small confirmation depth, while higher-value resources should wait for stronger confirmation before release.

Implementations should also make each payment usable only for the intended resource. A server should record the tuple $(pay_id, resource_id)$ and check it before forwarding the payment to the facilitator. This prevents the same payment proof from being reused to obtain multiple grants or to access a different resource. To avoid inconsistent behavior across SDKs, the ecosystem should also converge on a single canonical payment encoding, such as EIP-712 or a fixed JSON serialization, so that all implementations verify the same signed content in the same way.

Finally, paid responses should not be cacheable by shared middleware or edge proxies. Payment-gated routes should emit `Cache-Control: no-store` or `private` by default and bypass intermediary caches where possible. Implementations should also keep structured audit logs, including `pay_id`, `resource_id`, `tx_hash`, and timestamp, so that duplicate grants, failed settlements, and user disputes can be investigated after the fact.

6.5 Limitations and Future Work

Our formal and empirical scope stops at the protocol/facilitator boundary: fully malicious resource servers ($\mathcal{A}_R$) and colluding R/F pairs are out of scope. Hardening x402 against those adversaries would likely require client-visible settlement receipts or independent on-chain verification, rather than facilitator trust alone.

For Attack IV, the main paper validates only E1 (metadata manipulation) and E2 (Sybil flooding). The remaining marketplace threats E3–E5, such as price/latency manipulation and facilitator collusion, remain open evaluation targets. We view these as future work, together with broader ecosystem conformance testing and integration with A2A/x402 agent frameworks.

7 Related Work

We situate our work in five research strands.

Micropayment and payment channels. Layer-two protocols [4, 9] and payment channels [5, 10] provide off-chain scaling with on-chain dispute resolution. Probabilistic micropayments [6, 11] reduce settlement overhead via randomized redemption. State-channel protocols (Lightning, Raiden) require on-chain deposits and off-chain signed updates, with security depending on time-bound revocation. Voucher-based models (e.g., Filecoin retrieval markets) use sequence-numbered off-chain vouchers redeemed on-chain.

x402 differs fundamentally: it couples HTTP semantics with per-request on-chain settlement via a one-shot signed payload. No off-chain channel state is maintained, which simplifies deployment but exposes the HTTP–blockchain boundary to replay, duplication, and timing inconsistencies absent from channel-based designs.

Web payment and monetization standards. The W3C Payment Request API [23] mediates checkout via the user agent, Web Monetization [24] targets continuous content micropayments, and Open Payments [25] defines wallet-addressed REST APIs with grant-based authorization. Unlike x402, none use HTTP 402 as a per-request handshake binding payment authorization to an application

⁴ <https://eips.ethereum.org/EIPS/eip-8004>

Table 7: Our x402 attacks vs. related attacks.

Target	Attack	①	②	③	④	⑤
Payment channel	Payout race [12]	–	✓	–	✓	F
Payment channel	Atomicity [13]	–	✓	–	✓	F
PoS/PoA	Reorg [14, 15]	–	✓	–	▲	E
Agent tools	Metadata steering [16–18]	–	–	–	▲	E
MCP	Tool poisoning [19–22]	–	–	–	▲	E
x402 (ours)	I: Facilitator race	–	✓	–	✓	E+F
x402 (ours)	II: Replay	✓	–	–	✓	E+F
x402 (ours)	III: Header/Proxy	–	–	✓	–	E
x402 (ours)	IV: Server selection	–	–	–	✓	E

✓direct/primary coverage. ▲partial or adjacent coverage. – not addressed.

① **Replay**. ② **Reorg/Time**. ③ **Proxy/HTTP**.

④ **Economic impact**. ⑤ **Evidence** (E empirical, F formal, E+F both).

request via a one-shot signed payload, which exposes a distinct HTTP–blockchain trust boundary.

API security. Prior work on API monetization studies behavioral breaking changes [8], agent-ready REST APIs [26], and shared storage exploitation [27]. x402 extends this by tying API access to on-chain payments, introducing failure modes at the HTTP–blockchain boundary not covered by traditional API security.

Cross-layer atomicity and temporal blockchain risk. Payment-channel work addresses synchrony assumptions [28, 29] and cross-layer atomicity [12, 13], while PoS attacks [30, 31], BSC finality failures [15], and reorg-resilient attestation [32] expose the gap between *included* and *finalized* transactions. x402’s optimistic execution creates an analogous temporal gap, which our Attack I quantifies via RGP_k and T_{gf} .

Agent tool-selection and MCP attack surfaces. Agent tool choice is highly sensitive to text-visible metadata: editing descriptions shifts usage [16], and ToolTweak [17] and Attractive Metadata Attack [18] automate such steering, with ToolCert [33] certifying robustness under deceptive injection. MCP-focused studies broaden the threat model to registry and execution-layer compromise [19–22, 34, 35]. Our Attack IV specializes this surface to x402’s Bazaar discovery layer, focusing on economic steering of server selection rather than arbitrary tool execution.

8 Conclusion

x402 is not yet fully secure by construction. We identified five practical attacks leading to either *unpaid-service* or *paid-but-denied* without much cost. We validated attacks experimentally and empirically and propose concrete mitigations.

References

- [1] Erik Reppel, Ronnie Caspers, Kevin Leffew, Danny Organ, Dan Kim, and Nemil Dalal. x402: An open standard for internet-native payments, 2025.
- [2] Yuanzhe Zhang, Yuexin Xiang, Yuchen Lei, Qin Wang, Tian Qiu, Yujing Sun, Spiridon Zarkov, Tsz Hon Yuen, Andreas Deppeler, Jiangshan Yu, et al. SoK: Blockchain agent-to-agent payments. *arXiv preprint arXiv:2604.03733*, 2026.
- [3] Google. Agent2agent (A2A) protocol. Accessible at <https://a2a-protocol.org/latest/>, 2025.
- [4] Lewis Gudgeon, Pedro Moreno-Sanchez, Stefanie Roos, Patrick McCorry, and Arthur Gervais. SoK: Layer-two blockchain protocols. In *International Conference on Financial Cryptography and Data Security (FC)*, pages 201–226. Springer, 2020.

- [5] Lukas Aumayr, Sri AravindaKrishnan Thyagarajan, Giulio Malavolta, Pedro Moreno-Sanchez, and Matteo Maffei. Sleepy channels: Bi-directional payment channels without watchtowers. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 179–192, 2022.
- [6] Ghada Almashaqbeh, Allison Bishop, and Justin Cappos. Microcash: Practical concurrent processing of micropayments. In *International Conference on Financial Cryptography and Data Security (FC)*, pages 227–244. Springer, 2020.
- [7] Yue Li, Lei Wang, Kaixuan Wang, Zhiqiang Yang, Ke Wang, Zhi Guan, and Jianbo Gao. A402: Binding cryptocurrency payments to service execution for agentic commerce. *arXiv preprint arXiv:2603.01179*, 2026.
- [8] Dhanushka Jayasuriya, Valerio Terragni, Jens Dietrich, and Kelly Blincoe. Understanding the impact of APIs behavioral breaking changes on client applications. *Proceedings of the ACM on Software Engineering*, 1(FSE):1238–1261, 2024.
- [9] Minfeng Qi, Qin Wang, Zhipeng Wang, Manvir Schneider, Tianqing Zhu, Shipping Chen, William Knottenbelt, and Thomas Hardjono. SoK: Bitcoin layer two (L2). *ACM Computing Surveys (CSUR)*, 58(3):1–37, 2025.
- [10] Zeta Avarikioti, Orfeas Stefanos Thyfronitis Litos, and Roger Wattenhofer. Cerberus channels: Incentivizing watchtowers for Bitcoin. In *International Conference on Financial Cryptography and Data Security (FC)*, pages 346–366. Springer, 2020.
- [11] Taisei Takahashi and Akira Otsuka. Probabilistic micropayments with transferability. *European Symposium on Research in Computer Security (ESORICS)*, 2021.
- [12] Ben Weintraub, Satwik Prabhu Kumble, Cristina Nita-Rotaru, and Stefanie Roos. Payout races and congested channels: A formal analysis of security in the lightning network. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2562–2576, 2024.
- [13] Di Wu, Shoupeng Ren, Yuman Bai, Lipeng He, Jian Liu, Wu Wen, Kui Ren, and Chun Chen. On the atomicity and efficiency of blockchain payment channels. *USENIX Security Symposium (USENIX Sec)*, 2025.
- [14] Xinrui Zhang, Rujia Li, Qin Wang, Qi Wang, and Sisi Duan. Time-manipulation attack: Breaking fairness against proof of authority aura. In *Proceedings of the ACM Web Conference (WWW)*, pages 2076–2086, 2023.
- [15] Rujia Li, Jingyuan Ding, Qin Wang, Keting Jia, Haibin Zhang, and Sisi Duan. Does finality gadget finalize your block? a case study of Binance consensus. *USENIX Security Symposium (USENIX Sec)*, pages 4109–4125, 2025.
- [16] Kazem Faghieh, Wenxiao Wang, Yize Cheng, Siddhant Bharti, Gaurang Sriraman, Sriram Balasubramanian, Parsa Hosseini, and Soheil Feizi. Tool preferences in agentic LLMs are unreliable. In *The Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 20965–20980, 2025.
- [17] Jonathan Sneh, Ruomei Yan, Jialin Yu, Philip Torr, Yarin Gal, Sunando Sengupta, Eric Sommerlade, Alasdair Paren, and Adel Bibi. ToolTweak: An attack on tool selection in llm-based agents. *arXiv preprint arXiv:2510.02554*, 2025.
- [18] Kanghua Mo, Li Hu, Yucheng Long, and Zhihao Li. Attractive metadata attack: Inducing LLM agents to invoke malicious tools. *arXiv preprint arXiv:2508.02110*, 2025.
- [19] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. MCPTox: A benchmark for tool poisoning attack on real-world MCP servers. *arXiv preprint arXiv:2508.14925*, 2025.
- [20] Ruiqi Li, Zhiqiang Wang, Yunhao Yao, and Xiang-Yang Li. MCP-ITP: An automated framework for implicit tool poisoning in MCP. *arXiv preprint arXiv:2601.07395*, 2026.
- [21] Dongsen Zhang, Zekun Li, Xu Luo, Xuannan Liu, Peipei Li, and Wenjun Xu. MCP security bench (MSB): Benchmarking attacks against model context protocol in LLM agents. *arXiv preprint arXiv:2510.15994*, 2025. Accepted to ICLR 2026.
- [22] Yohan Lee, Jisoo Jang, Seoyeon Choi, Sangyeop Kim, and Seungtaek Choi. Overthinking loops in agents: A structural risk via MCP tools. *arXiv preprint arXiv:2602.14798*, 2026.
- [23] World Wide Web Consortium. Payment request API, 2022. W3C Recommendation. Available at <https://www.w3.org/TR/2022/REC-payment-request-20220908/>.
- [24] Web Platform Incubator Community Group. Web monetization, 2025. Draft Community Group Report, 13 March 2025. Available at <https://webmonetization.org/specification>.
- [25] Open Payments. Overview, n.d. Open Payments (1.2.0) resource server documentation; accessed April 23, 2026.
- [26] Meriem Mastouri, Emna Ksontini, Amine Barrak, and Wael Kessentini. From REST to MCP: An empirical study of API wrapping and automated server generation for LLM agents. *arXiv preprint arXiv:2507.16044*, 2025.
- [27] Alexandra Nisenoff, Deian Stefan, and Nicolas Christin. Exploiting the shared storage API. *ACM Conference on Computer and Communications Security (CCS)*, 2025.
- [28] Zeta Avarikioti, Eleftherios Kokoris-Kogias, Roger Wattenhofer, and Dionysis Zindros. Brick: Asynchronous incentive-compatible payment channels. In *International Conference on Financial Cryptography and Data Security (FC)*, pages 209–230. Springer, 2021.

- [29] Oguzhan Ersoy, Jérémie Decouchant, Satwik Prabhu Kumble, and Stefanie Roos. Syncpcn/psyncpcn: Payment channel networks without blockchain synchrony. In *ACM Conference on Advances in Financial Technologies (AFT)*, 2022.
- [30] Caspar Schwarz-Schilling, Joachim Neu, Barnabé Monnot, Aditya Asgaonkar, Ertem Nusret Tas, and David Tse. Three attacks on proof-of-stake Ethereum. In *International Conference on Financial Cryptography and Data Security (FC)*, pages 560–576. Springer, 2022.
- [31] Mingfei Zhang, Rujia Li, and Sisi Duan. Max attestation matters: Making honest parties lose their incentives in Ethereum PoS. In *USENIX Security Symposium (USENIX Sec)*, pages 6255–6272, 2024.
- [32] Mingfei Zhang, Rujia Li, Xueqian Lu, and Sisi Duan. Available attestation: Towards a reorg-resilient solution for Ethereum proof-of-stake. *USENIX Security Symposium (USENIX Sec)*, 2025.
- [33] Jehyeok Yeon, Isha Chaudhary, and Gagandeep Singh. Quantifying distributional robustness of agentic tool-selection. *arXiv preprint arXiv:2510.03992*, 2025.
- [34] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (MCP): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*, 2025.
- [35] Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, et al. SoK: Agentic skills—beyond tool use in LLM agents. *arXiv preprint arXiv:2602.20867*, 2026.

Open Science

We host our evaluation artifacts at <https://anonymous.4open.science/r/x402-attack-FDF1>.

A Full Proofs of Security Theorems

We show the full proofs for the theorem statements in §2.5.

A.1 Proof of Theorem 6

Theorem 6 presents the theorem of *Authorization Soundness-Conservative Execution*.

PROOF OF THEOREM 6. We prove that under conservative execution, any grant issued by the resource server must correspond to a payment transaction that has already reached depth k in the blockchain view of an honest facilitator. Therefore, any authorization failure after the grant necessarily implies a post-grant chain reorganization of depth at least k , which is bounded by the chain finality error $\epsilon_{\text{chain}}(k)$. We instantiate this bound under exponential finality decay to obtain negligibility in the security parameter.

Step 1: Grant implies k -deep confirmation at grant time. Under conservative execution, R issues $R.\text{grant}(C, \text{res}, t_g)$ only when F reports $\text{Confirmations}(tx_{pp}, t_g) \geq k$. Since F is honest (Assumption 2.3), this report faithfully reflects F 's view of $\mathcal{B}$: at time t_g , tx_{pp} has been included in some block b at depth $\geq k$ in the chain known to F . Formally, let $\text{Depth}(tx, t)$ denote the depth of tx 's containing block in the canonical chain at time t . Then:

$$R.\text{grant}(C, \text{res}, t_g) \Rightarrow \text{Depth}(tx_{pp}, t_g) \geq k. \quad (\dagger)$$

Step 2: E_{auth} requires a post-grant reorganization. E_{auth} occurs when tx_{pp} eventually becomes absent from the canonical chain despite the grant at t_g . By $(\dagger)$, tx_{pp} was at depth $\geq k$ at time t_g . For tx_{pp} to later become absent, the chain must undergo a reorganization of depth $\geq k$ at some time $t' > t_g$:

$$E_{\text{auth}} \subseteq \{\exists t' > t_g : \text{Reorg}(\text{depth} \geq k, t')\}.$$

Step 3: Bounding the reorganization probability. By Assumption 2.3, the event $\text{Reorg}(\text{depth} \geq k, t')$ is independent of the grant event conditional on the broadcast of tx_{pp} , and in particular is not influenced by F 's reporting behavior or HTTP-layer scheduling.

Therefore:

$$\begin{aligned} \Pr[E_{\text{auth}}] &\leq \Pr[\exists t' > t_g : \text{Reorg}(\text{depth} \geq k, t')] \\ &\leq \Pr[\text{Reorg}(\text{depth} \geq k)] \leq \epsilon_{\text{chain}}(k), \end{aligned}$$

where the last inequality is Definition 1.

Step 4: Parameter choice for negligibility. For exponential decay $\epsilon_{\text{chain}}(k) = e^{-\alpha k}$, set $k^* = \lceil \alpha^{-1}(\lambda + 1) \ln 2 \rceil$. Then $e^{-\alpha k^*} \leq e^{-(\lambda+1) \ln 2} = 2^{-(\lambda+1)} < 2^{-\lambda}$, which is negligible in λ . □

A.2 Proof of Theorem 7

Theorem 7 refers to the theorem of *Authorization Soundness Violation-Optimistic Execution*.

PROOF OF THEOREM 7. Let t_0 be the time C broadcasts tx_{pp} to the mempool. Define the events

$$\text{VulnWindow} = \{t_g < t_0 + T_{\text{inc}} + k \cdot T_b\},$$

$$\text{Reorg}_k = \{\mathcal{A}_{\mathcal{B}} \text{ causes a reorg during } [t_g, t_0 + T_{\text{inc}} + k \cdot T_b]\}.$$

Step 1: Characterize the grant time. Under optimistic execution, the resource server grants at $t_g = t_0 + T_{\text{verify}} + \delta$. The grant precedes k -confirmation finality exactly when

$$t_0 + T_{\text{verify}} + \delta < t_0 + T_{\text{inc}} + k \cdot T_b,$$

which is equivalent to $T_{\text{inc}} + k \cdot T_b > T_{\text{verify}} + \delta$.

Step 2: Revert-grant requires both a vulnerable window and a reorganization. If a grant occurs before finality and the transaction is later removed from the canonical chain, then the execution contains both VulnWindow and Reorg_k . Hence $E_{\text{revert}} \supseteq \text{VulnWindow} \cap \text{Reorg}_k$.

Step 3: Apply independence. By Assumption 2.3, the chain reorganization process is independent of HTTP timing once the set of broadcast transactions is fixed. Therefore

$$\begin{aligned} \text{RGP}_k &= \Pr[E_{\text{revert}}] \\ &\geq \Pr[\text{VulnWindow} \cap \text{Reorg}_k] \\ &= \Pr[\text{Reorg}_k] \cdot \Pr[\text{VulnWindow}] \\ &\geq p_{\text{reorg}} \cdot \Pr[T_{\text{inc}} + k \cdot T_b > T_{\text{verify}} + \delta]. \end{aligned}$$

Step 4: Special case $k = 0$. Setting $k = 0$ yields

$$\text{RGP}_0 \geq p_{\text{reorg}} \cdot \Pr[T_{\text{inc}} > T_{\text{verify}} + \delta].$$

□

A.3 Proof of Theorem 8

Theorem 8 refers to the *Replay Resistance under Pre-Grant Uniqueness Enforcement*.

PROOF OF THEOREM 8. We proceed with two directions.

Positive direction. Let t be the time R first accepts PP with identifier pay_id . The server records $\mathcal{I}[\text{pay_id}] \leftarrow (\text{resource_id}, t)$.

Case 1: Replay within the TTL window. For any replay arriving at time $t' \in (t, t + \text{TTL}]$, the lookup in $\mathcal{I}$ occurs before any further verification or grant. Because the entry has not expired, the lookup succeeds and the request is rejected before access is granted. Hence E_{replay} cannot occur in this interval.

Case 2: Replay after the TTL window. Once the $\mathcal{I}$ entry expires, the timestamp defense applies. The embedded timestamp satisfies $ts \approx t$, while the server rejects any payload with $ts \notin [t' - W, t' + W]$. Since $t' > t + \text{TTL}$ and $W < \text{TTL}$, we have $t < t' - W$, so $ts < t' - W$, causing rejection by the freshness check. Thus the theorem's stated guarantee on $t' \in (t, t + \text{TTL}]$ holds, and the combined defenses are in fact stronger.

Negative direction. Without $\mathcal{I}$, each replayed request is processed independently by R . The resource server forwards $\langle PP, PR \rangle$ to F ; the facilitator verifies the still-valid signature and returns an approval, so each request triggers a fresh grant. If this is repeated n times, then

$$\text{GrantCount}(\text{pay_id}, \mathcal{E}) = n$$

with probability 1. $\square$

A.4 Proof of Theorem 9

Theorem 9 presents the *Facilitator k -Atomicity as Prerequisite*.

PROOF OF THEOREM 9. We prove that facilitator k -atomicity is necessary for authorization soundness under conservative execution. If F may report finality before tx_{pp} actually reaches depth k , then the server can grant access based on an under-confirmed transaction. A later reorganization can therefore invalidate the payment after the grant, yielding a non-negligible authorization failure probability.

Step 1: Non-atomicity induces effective under-confirmation. When F is not k -atomic, there exist executions in which F reports $\text{Final}(tx_{pp}, k)$ at time t_g while $\text{Confirmations}(tx_{pp}, t_g) = k' < k$. Under conservative execution, R grants access upon this report, so the effective confirmation depth at grant time is k' , not k .

Step 2: A subsequent reorganization can invalidate the grant. For tx_{pp} at depth $k' < k$, a reorganization of depth $\geq k' + 1 \leq k$ suffices to remove tx_{pp} from the canonical chain. By Assumption 2.3, this reorganization event depends only on the chain process, not on F 's reporting behavior.

Step 3: Independence argument. Let $\mathcal{G}$ be the σ -algebra generated by HTTP-layer events and facilitator reports up to time t_g . The event E_{atom} is $\mathcal{G}$ -measurable, while the post-grant reorganization event is determined by the mining process after t_g . Assumption 2.3 makes these events independent.

Step 4: Combine the two events. Authorization soundness fails whenever F prematurely reports finality and the chain later reorganizes deeply enough to remove tx_{pp} . Therefore

$$\begin{aligned} \Pr[E_{\text{auth}}] &\geq \Pr[E_{\text{atom}} \wedge \text{Reorg}(\text{depth} \geq k' + 1)] \\ &= \Pr[E_{\text{atom}}] \cdot \Pr[\text{Reorg}(\text{depth} \geq k' + 1)] \\ &\geq \mu \cdot p_{\text{reorg}}. \end{aligned}$$

Since μ is non-negligible and $p_{\text{reorg}} > 0$, the product is non-negligible violating authorization soundness. $\square$

A.5 Proof of Corollary 10

Corollary 10 shows the *Confirmation Depth Recommendation*.

PROOF OF COROLLARY 10. The first statement is immediate from Theorem 6: under conservative execution with an honest facilitator, any choice of k guarantees

$$\Pr[E_{\text{auth}}] \leq \epsilon_{\text{chain}}(k).$$

To meet a target failure probability ϵ_{target} , it therefore suffices to choose the smallest confirmation threshold satisfying $\epsilon_{\text{chain}}(k) \leq \epsilon_{\text{target}}$, which is exactly the definition of k^* .

For the Base-style exponential calibration $\epsilon_{\text{chain}}(k) \approx e^{-\alpha k}$, this becomes

$$k^* = \min\{k : e^{-\alpha k} \leq \epsilon_{\text{target}}\} = \lceil \alpha^{-1} \ln(\epsilon_{\text{target}}^{-1}) \rceil.$$

Instantiating this expression at the two target risk levels used in the paper, $\epsilon_{\text{target}} = 10^{-2}$ and $\epsilon_{\text{target}} = 10^{-4}$, yields the illustrative recommendations $k \geq 3$ and $k \geq 12$ under the chain-specific Base calibration assumed in the corollary statement. $\square$